

Mini-Project 3: Polymorphism and Type Inference

CAS CS 320: Concepts of Programming Languages

Due Thursday 4/30 by 8:00PM

1 Introduction

We're on our last interpreter. In this one, we're adding polymorphism and Hindley-Milner(light)-style type inference. You're required to implement the following functions according to the specification below.

```
val type_of_expr : ctxt -> expr -> (ty_scheme, Error_msg.t)
val eval_expr : dyn_env -> expr -> value
```

Your implementations of the above functions should appear in the file `interp3/lib/interp3.ml`. This mini-project has a great number of similarities to the previous one; as a result the spec is less generally verbose. A couple of notes on what's different.

- This interpreter has polymorphism and Hindley-Milner-light-style type inference. This means that each top-level-let-defined expression is given its principal polymorphic type, and that local-lets are not polymorphic.
- It includes user-defined algebraic data types. You do not need to worry about user-defined `type` declarations, but you will have to deal with arbitrary constructors according to the rules below.
- We introduced a wildcard pattern—distinct from a variable—to eliminate some of the confusion about wildcards from the previous mini-project.
- Expressions are preprocessed further than in the previous mini-project. This will make some of the implementation details simpler (e.g., there are no multi-argument functions).
- This interpreter includes strings, as well as `print_endline`.

Please read through the entire spec before continuing to the code.

2 Syntax

A program in our language is given by the following grammar. We've implemented the parser for you, but you still need to understand how this grammar works and how it corresponds to the types `ty`, `pattern`, `bop`, `expr`, and `stmt`. This grammar represents the *concrete* syntax of the language. The *abstract* syntax has been simplified for this mini-project, e.g., see the definition of `expr`.

```
<prog> ::= {<stmt>}
<stmt> ::= let <var> {<arg>} [: <ty>] = <expr>
          | let rec <var> <arg> {<arg>} [: <ty>] = <expr>
          | type [<tpars>] <var> = <constrs>
<arg> ::= <var> | ( <var> : <ty> )
<expr> ::= () | true | false | <int> | <string> | - <expr> | <expr> <op> <expr>
          | if <expr> then <expr> else <expr>
          | ( <expr> : <ty> ) | <expr> , <expr> { , <expr> } | assert <expr> | <var>
          | [] | [<expr> { ; <expr> } ; ] | <expr> :: <expr> | <cons> [<expr>]
          | fun <arg> {<arg>} -> <expr> | <expr> <expr> {<expr>}
          | let <var> {<arg>} [: <ty>] = <expr> in <expr>
          | let rec <var> <arg> {<arg>} [: <ty>] = <expr> in <expr>
          | match <expr> with | <pat> -> <expr> { | <pat> -> <expr> }
<pat> ::= _ | <var> | () | true | false | print.endline | <int> | <pat> , <pat> { , <pat> }
          | [] | [<pat> { ; <pat> } ; ] | <pat> :: <pat> | <cons> [<pat>] | ( <pat> )
<op> ::= + | - | * | / | mod | = | <> | < | <= | > | >= | && | || | :: | ^
<ty> ::= unit | bool | int | string | <ty> * <ty> { * <ty> } | [<tys>] <var> | <ty> -> <ty> | <tpar> | ( <ty> )
<tpars> ::= <tpar> | ( <tpar> , <tpar> { , <tpar> } )
<tys> ::= <ty> | ( <ty> , <ty> { , <ty> } )
```

3 Typing

In this mini-project we're using a constraint-based inference system. The rules are defined over judgments of the form " $\Gamma \vdash e : \tau \dashv \mathcal{C}$ " which are read as "the expression e has type τ in the context Γ relative to the unification constraints in $\mathcal{C}$." The only new constructs you need to consider are strings and user-defined constructors.

Literals

$$\begin{array}{c} \frac{}{\Gamma \vdash () : \text{unit} \dashv \emptyset} \text{UNIT} \qquad \frac{}{\Gamma \vdash \text{true} : \text{bool} \dashv \emptyset} \text{TRUE} \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool} \dashv \emptyset} \text{FALSE} \\[10pt] \frac{\text{n is an integer literal}}{\Gamma \vdash n : \text{int} \dashv \emptyset} \text{INTLIT} \qquad \frac{\text{s is a string literal}}{\Gamma \vdash s : \text{string} \dashv \emptyset} \text{STRING} \end{array}$$

Arithmetic Operators

$$\begin{array}{c} \frac{\Gamma \vdash e : \tau \dashv \mathcal{C}}{\Gamma \vdash -e : \text{int} \dashv \tau \doteq \text{int}, \mathcal{C}} \text{NEGATE} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 + e_2 : \text{int} \dashv \tau_1 \doteq \text{int}, \tau_2 \doteq \text{int}, \mathcal{C}_1, \mathcal{C}_2} \text{ADD} \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 - e_2 : \text{int} \dashv \tau_1 \doteq \text{int}, \tau_2 \doteq \text{int}, \mathcal{C}_1, \mathcal{C}_2} \text{SUB} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 * e_2 : \text{int} \dashv \tau_1 \doteq \text{int}, \tau_2 \doteq \text{int}, \mathcal{C}_1, \mathcal{C}_2} \text{MUL} \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 / e_2 : \text{int} \dashv \tau_1 \doteq \text{int}, \tau_2 \doteq \text{int}, \mathcal{C}_1, \mathcal{C}_2} \text{DIV} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 \text{ mod } e_2 : \text{int} \dashv \tau_1 \doteq \text{int}, \tau_2 \doteq \text{int}, \mathcal{C}_1, \mathcal{C}_2} \text{MOD} \end{array}$$

Boolean Operators

$$\begin{array}{c} \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 \&\& e_2 : \text{bool} \dashv \tau_1 \doteq \text{bool}, \tau_2 \doteq \text{bool}, \mathcal{C}_1, \mathcal{C}_2} \text{AND} \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 || e_2 : \text{bool} \dashv \tau_1 \doteq \text{bool}, \tau_2 \doteq \text{bool}, \mathcal{C}_1, \mathcal{C}_2} \text{OR} \end{array}$$

String Operators

$$\begin{array}{c} \frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 \wedge e_2 : \text{string} \dashv \tau_1 \doteq \text{string}, \tau_2 \doteq \text{string}, \mathcal{C}_1, \mathcal{C}_2} \text{CONCAT} \\[10pt] \frac{}{\emptyset \vdash \text{print_newline} : \text{string} \rightarrow \text{unit} \dashv \emptyset} \text{PRINT} \end{array}$$

The `print\_newline` function is introduced in the context at the start of interpretation.

Comparison Operators

$$\begin{array}{c}
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 = e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{EQ} \\
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 < e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{LT} \\
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 > e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{GT} \\
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 <> e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{NEQ} \\
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 \leq e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{LEQ} \\
\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 \geq e_2 : \text{bool} \dashv \tau_1 \doteq \tau_2, \mathcal{C}_1, \mathcal{C}_2} \text{GEQ}
\end{array}$$

Conditionals

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2 \quad \Gamma \vdash e_3 : \tau_3 \dashv \mathcal{C}_3}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau_2 \dashv \tau_1 \doteq \tau_3, \mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3} \text{IF}$$

Type Annotations

$$\frac{\Gamma \vdash e : \tau' \dashv \mathcal{C}}{\Gamma \vdash (e : \tau) : \tau \dashv \tau' \doteq \tau, \mathcal{C}} \text{ANNOT}$$

Tuples

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \dots \quad \Gamma \vdash e_k : \tau_k \dashv \mathcal{C}_k}{\Gamma \vdash e_1, \dots, e_k : \tau_1 * \dots * \tau_k \dashv \mathcal{C}_1, \dots, \mathcal{C}_k} \text{TUPLE}$$

Assertions

$$\frac{\alpha \text{ is fresh}}{\Gamma \vdash \text{assert false} : \alpha \dashv \emptyset} \text{ASSERTFALSE} \quad \frac{\Gamma \vdash e : \tau \dashv \mathcal{C}}{\Gamma \vdash \text{assert } e : \text{unit} \dashv \tau \doteq \text{bool}, \mathcal{C}} \text{ASSERT}$$

Take careful note of the ASSERTFALSE rule. For this rule (and the ones that follow), you should use the function `val fresh : unit -> ty` provided in the starter code to generate new type parameters.

Variables

$$\frac{(x : \forall \alpha_1 \dots \forall \alpha_k. \tau) \in \Gamma \quad \beta_1, \dots, \beta_k \text{ are fresh}}{\Gamma \vdash x : [\beta_k / \alpha_k] \dots [\beta_1 / \alpha_1] \tau \dashv \emptyset} \text{VAR}$$

Constructors

$$\frac{(C : \forall \alpha_1 \dots \forall \alpha_k. (\alpha_1, \dots, \alpha_k) \text{ adt_name}) \in \Gamma \quad \beta_1, \dots, \beta_k \text{ are fresh}}{\Gamma \vdash C : (\beta_1, \dots, \beta_n) \text{ adt_name} \dashv \emptyset} \text{CONS0} \\
\frac{(C : \forall \alpha_1 \dots \forall \alpha_k. \sigma \rightarrow (\alpha_1, \dots, \alpha_k) \text{ adt_name}) \in \Gamma \quad \Gamma \vdash e : \tau \dashv \mathcal{C} \quad \beta_1, \dots, \beta_k \text{ are fresh}}{\Gamma \vdash C e : (\beta_1, \dots, \beta_n) \text{ adt_name} \dashv \tau \doteq [\beta_k / \alpha_k] \dots [\beta_1 / \alpha_1] \sigma, \mathcal{C}} \text{CONS1}$$

In these rules, `adt_name` represents the name of an algebraic data type defined by the user and `C` represents the name of one of its constructors. You will have to verify that the name of the constructor appears as an identifier in the context. Note that variable names and constructor names cannot clash because constructor names must begin with a capital letter.

Also note that we don't have explicit rules for lists. Instead we take $x :: xs$ to be shorthand for `Cons(x, xs)` and `oci[]` for `Nil`, both of which will be included in the context before interpretation.

Functions

$$\frac{\alpha \text{ is fresh} \quad \Gamma, x : \alpha \vdash e : \tau \dashv \mathcal{C}}{\Gamma \vdash \text{fun } x \rightarrow e : \alpha \rightarrow \tau \dashv \mathcal{C}} \text{FUN} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \dashv \mathcal{C}}{\Gamma \vdash \text{fun } (x : \tau_1) \rightarrow e : \tau_1 \rightarrow \tau_2 \dashv \mathcal{C}} \text{FUNANNOT}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_1 \quad \alpha \text{ is fresh}}{\Gamma \vdash e_1 e_2 : \alpha \dashv \tau_1 \doteq \tau_2 \rightarrow \alpha, \mathcal{C}_1, \mathcal{C}_2} \text{APP}$$

Let-Expressions

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2 \dashv \mathcal{C}_1, \mathcal{C}_2} \text{LET}$$

$$\frac{\alpha \text{ is fresh} \quad \Gamma, f : \alpha \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \quad \Gamma, f : \tau_1 \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash \text{let rec } f = e_1 \text{ in } e_2 : \tau_2 \dashv \alpha \doteq \tau_1, \mathcal{C}_1, \mathcal{C}_2} \text{LETREC}$$

Again, this does *not* allow for local let-polymorphism, which is a departure from the way that OCaml behaves. Also note that recursive let-expressions are syntactically required to have at least one argument. The syntactic preprocessing that we makes the argument into part of an anonymous function.

Pattern Typing

The following rules are nearly identical to those given in the previous project, except that pattern typing now requires the context (for ADT constructors), and can generate constraints.

$$\frac{\alpha \text{ is fresh}}{\Gamma \vdash _ : \alpha \dashv \emptyset \Rightarrow \emptyset} \text{WILDP} \qquad \frac{\alpha \text{ is a variable pattern} \quad \alpha \text{ is fresh}}{\Gamma \vdash x : \alpha \dashv \emptyset \Rightarrow \{x : \alpha\}} \text{VARP} \qquad \frac{}{\Gamma \vdash () : \text{unit} \dashv \emptyset \Rightarrow \emptyset} \text{UNITP}$$

$$\frac{}{\Gamma \vdash \text{true} : \text{bool} \dashv \emptyset \Rightarrow \emptyset} \text{TRUEP} \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool} \dashv \emptyset \Rightarrow \emptyset} \text{FALSEP}$$

$$\frac{n \text{ is an integer literal pattern}}{\Gamma \vdash n : \text{int} \dashv \emptyset \Rightarrow \emptyset} \text{INTLITP} \qquad \frac{s \text{ is a string literal pattern}}{\Gamma \vdash s : \text{string} \dashv \emptyset \Rightarrow \emptyset} \text{STRINGLITP}$$

$$\frac{\Gamma \vdash p_1 : \tau_1 \dashv \mathcal{C}_1 \Rightarrow \Gamma_1 \quad \dots \quad \Gamma \vdash p_k : \tau_k \dashv \mathcal{C}_k \Rightarrow \Gamma_k \quad \Gamma_1, \dots, \Gamma_k \text{ are disjoint}}{\Gamma \vdash p_1, \dots, p_k : \tau_1 * \dots * \tau_k \dashv \mathcal{C}_1, \dots, \mathcal{C}_k \Rightarrow \Gamma_1, \dots, \Gamma_k} \text{TUPLEP}$$

$$\frac{(C : \forall \alpha_1 \dots \forall \alpha_k. (\alpha_1, \dots, \alpha_k) \text{ adt_name}) \in \Gamma \quad \beta_1, \dots, \beta_k \text{ are fresh}}{\Gamma \vdash C : (\beta_1, \dots, \beta_n) \text{ adt_name} \dashv \emptyset} \text{CONSP0}$$

$$\frac{(C : \forall \alpha_1 \dots \forall \alpha_k. \sigma \rightarrow (\alpha_1, \dots, \alpha_k) \text{ adt_name}) \in \Gamma \quad \Gamma \vdash p : \tau \dashv \mathcal{C} \Rightarrow \Gamma' \quad \beta_1, \dots, \beta_k \text{ are fresh}}{\Gamma \vdash C e : (\beta_1, \dots, \beta_n) \text{ adt_name} \dashv \tau \doteq [\beta_k / \alpha_k] \dots [\beta_1 / \alpha_1] \sigma, \mathcal{C} \Rightarrow \Gamma'} \text{CONSP1}$$

Match-Expressions

Note that the process is a bit different here; instead of checking if a pattern matches the type of the matched expression, we infer the type of the pattern and then constrain it to be the same as the type of the matched expression.

$$\frac{\Gamma \vdash e : \tau \dashv \mathcal{C} \quad \Gamma \vdash p_1 : \tau_1 \dashv \mathcal{C}_1 \Rightarrow \Gamma_1 \quad \Gamma, \Gamma_1 \vdash e_1 : \tau'_1 \dashv \mathcal{C}'_1 \quad \dots \quad \Gamma \vdash p_k : \tau_k \dashv \mathcal{C}_k \Rightarrow \Gamma_k \quad \Gamma, \Gamma_k \vdash e_k : \tau'_k \dashv \mathcal{C}'_k}{\Gamma \vdash \text{match } e \text{ with } | p_1 \text{ -> } e_1 \dots | p_k \text{ -> } e_k : \tau'_1 \dashv \tau_1 \doteq \tau, \dots, \tau_k \doteq \tau, \tau'_2 \doteq \tau'_1, \dots, \tau'_k \doteq \tau'_1, \mathcal{C}, \mathcal{C}_1, \mathcal{C}'_1, \dots, \mathcal{C}_k, \mathcal{C}'_k} \text{MATCH}$$

For a given context Γ and expression e , once you've determined τ and $\mathcal{C}$ such that $\Gamma \vdash e : \tau \dashv \mathcal{C}$, you have to:

- solve the unification problem defined by $\mathcal{C}$ to get a most general unifier $\mathcal{S}$;
- Apply $\mathcal{S}$ and τ and generalize over the free variables of $\mathcal{S}\tau$ to get the *principal type* of e (which is technically a *type scheme*).
- Normalize the type parameter names according to ocaml standards, i.e., parameter names should be given alphabetical order according to first appearance in the type, e.g., you should provide the type

```
'a 'b . 'a -> 'b -> 'a
```

and not

```
'b 'a . 'b -> 'a -> 'b
```

As an `ty` value, you need to return:

```
[TParam "a"; TParam "b"], TFun (TParam "a" (TFun (TParam "b", TParam "a")))
```

4 Semantics

The notion of evaluation used in this mini-project is almost identical to the one used in the previous mini-project, and there are only a few new constructs. Consequently, there is little exposition in this section. The rules are still defined over judgments of the form “ $\mathcal{E} \vdash e \Downarrow v$ ” which read “ e evaluates to v in the environment $\mathcal{E}$ ”. The major difference between this mini-project and the previous one with respect to semantics is pattern matching. We take a **value** (i.e., something of type `value`) to be one of the following sorts of things:

- a unit, denoted $\bullet$;
- a Boolean value, either the true value, denoted $\top$, or the false value, denoted $\perp$;
- an integer (i.e., an element of the set $\mathbb{Z}$) denoted $1, 2, -23$, etc.;
- a string denote, e.g., `"abc123"`.
- a constructor value, denoted by the italicized name of the constructor, e.g., the constructor `Foo` and `Bar 2` are represented by the constructor values *Foo* and *Bar*(2).
- a tuple value, denoted $(v_1, \dots, v_k)$ for values $v_1, \dots, v_k$.
- a closure, denoted $\langle \mathcal{E}, x \mapsto e \rangle$, which consists of an environment $\mathcal{E}$, an optional name x , and an expression e ; we write $\langle \mathcal{E}, \cdot \mapsto e \rangle$ for a closure with no name;

Literals

$$\begin{array}{c}
 \frac{}{\mathcal{E} \vdash () \Downarrow \bullet} \text{UNITE} \quad \frac{}{\mathcal{E} \vdash \text{true} \Downarrow \top} \text{TRUEE} \quad \frac{}{\mathcal{E} \vdash \text{false} \Downarrow \perp} \text{FALSEE} \quad \frac{\text{n is an integer literal for } n}{\mathcal{E} \vdash n \Downarrow n} \text{INTLITE} \\
 \frac{\text{s is a string literal for s}}{\mathcal{E} \vdash s \Downarrow s} \text{STRINGE}
 \end{array}$$

Arithmetic Operators

$$\begin{array}{c}
 \frac{\mathcal{E} \vdash e \Downarrow n}{\mathcal{E} \vdash -e \Downarrow -n} \text{NEGATEE} \quad \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2}{\mathcal{E} \vdash e_1 + e_2 \Downarrow v_1 + v_2} \text{ADDINTE} \quad \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2}{\mathcal{E} \vdash e_1 - e_2 \Downarrow v_1 - v_2} \text{SUBINTE} \\
 \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2}{\mathcal{E} \vdash e_1 * e_2 \Downarrow v_1 v_2} \text{MULINTE} \quad \frac{\mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_2 \neq 0 \quad \mathcal{E} \vdash e_1 \Downarrow v_1}{\mathcal{E} \vdash e_1 / e_2 \Downarrow v_1 / v_2} \text{DIVINTE} \\
 \frac{\mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_2 \neq 0 \quad \mathcal{E} \vdash e_1 \Downarrow v_1}{\mathcal{E} \vdash e_1 \text{ mod } e_2 \Downarrow v_1 \text{ (mod } v_2)} \text{MODINTE}
 \end{array}$$

Boolean Operators

$$\begin{array}{c}
 \frac{\mathcal{E} \vdash e_1 \Downarrow \top \quad \mathcal{E} \vdash e_2 \Downarrow v}{\mathcal{E} \vdash e_1 \&\& e_2 \Downarrow v} \text{ANDTRUE} \quad \frac{\mathcal{E} \vdash e_1 \Downarrow \perp}{\mathcal{E} \vdash e_1 \&\& e_2 \Downarrow \perp} \text{ANDFALSE} \quad \frac{\mathcal{E} \vdash e_1 \Downarrow \top}{\mathcal{E} \vdash e_1 || e_2 \Downarrow \top} \text{ORTRUE} \\
 \frac{\mathcal{E} \vdash e_1 \Downarrow \perp \quad \mathcal{E} \vdash e_2 \Downarrow v}{\mathcal{E} \vdash e_1 || e_2 \Downarrow v} \text{ORFALSE}
 \end{array}$$

String Operators

In the following rule, the operator “o” stands for string concatenation.

$$\frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2}{\mathcal{E} \vdash e_1 \mathbin{\text{^}} e_2 \Downarrow v_1 \circ v_2} \text{CONCAT}$$

We don’t include an evaluation rule for `print_endline`. We simply treat this as a variable whose value appears in the dynamic environment. **You still have to implement it in your semantics if you want to be able to use it for testing.** This requires you to treat the `print_endline` closures as a special case. That said, we will not test `print_endline`.¹

Comparison Operators

$$\begin{array}{ll} \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 = v_2}{\mathcal{E} \vdash e_1 = e_2 \Downarrow \top} \text{EQTRUE} & \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \neq v_2}{e_1 = e_2 \Downarrow \perp} \text{EQFALSE} \\ \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 \neq v_2}{\mathcal{E} \vdash e_1 < e_2 \Downarrow \top} \text{NEQTRUE} & \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 = v_2}{\mathcal{E} \vdash e_1 < e_2 \Downarrow \perp} \text{NEQFALSE} \\ \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 < v_2}{\mathcal{E} \vdash e_1 < e_2 \Downarrow \top} \text{LTTRUE} & \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 \geq v_2}{\mathcal{E} \vdash e_1 < e_2 \Downarrow \perp} \text{LTFALSE} \\ \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 \leq v_2}{\mathcal{E} \vdash e_1 <= e_2 \Downarrow \top} \text{LEQTRUE} & \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 > v_2}{\mathcal{E} \vdash e_1 <= e_2 \Downarrow \perp} \text{LEQFALSE} \\ \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 > v_2}{\mathcal{E} \vdash e_1 > e_2 \Downarrow \top} \text{GTTRUE} & \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 \leq v_2}{\mathcal{E} \vdash e_1 > e_2 \Downarrow \perp} \text{GTFALSE} \\ \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 \geq v_2}{\mathcal{E} \vdash e_1 >= e_2 \Downarrow \top} \text{GEQTRUE} & \frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \mathcal{E} \vdash e_2 \Downarrow v_2 \quad v_1 < v_2}{\mathcal{E} \vdash e_1 >= e_2 \Downarrow \perp} \text{GEQFALSE} \end{array}$$

Conditionals

$$\frac{\mathcal{E} \vdash e_1 \Downarrow \top \quad \mathcal{E} \vdash e_2 \Downarrow v}{\mathcal{E} \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v} \text{IFTRUE} \quad \frac{\mathcal{E} \vdash e_1 \Downarrow \perp \quad \mathcal{E} \vdash e_3 \Downarrow v}{\mathcal{E} \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v} \text{IFFALSE}$$

Type Annotations

$$\frac{\mathcal{E} \vdash e \Downarrow v}{\mathcal{E} \vdash (e : \tau) \Downarrow v} \text{ANNOTATE}$$

Tuples

$$\frac{\mathcal{E} \vdash e_1 \Downarrow v_1 \quad \dots \quad \mathcal{E} \vdash e_k \Downarrow v_k}{\mathcal{E} \vdash e_1 , \dots , e_k \Downarrow (v_1, \dots, v_k)} \text{TUPLEE}$$

¹In a sense we can’t.

Assertions

$$\frac{\mathcal{E} \vdash e \Downarrow \top}{\mathcal{E} \vdash \text{assert } e \Downarrow \bullet} \text{ ASSERTE}$$

Variables

$$\frac{}{\mathcal{E} \vdash x \Downarrow \mathcal{E}(x)} \text{ VARE}$$

Constructors

$$\frac{\text{C is a constructor C}}{\mathcal{E} \vdash C \Downarrow C} \text{ CONS0E} \quad \frac{\text{C is a constructor C} \quad \mathcal{E} \vdash e \Downarrow v}{\mathcal{E} \vdash C e \Downarrow C(v)} \text{ CONS1E}$$

Functions

$$\frac{}{\mathcal{E} \vdash \text{fun } x \rightarrow e \Downarrow (\lambda \mathcal{E}, \cdot \mapsto \text{fun } x \rightarrow e)} \text{ FUNE}$$

$$\frac{\mathcal{E}_1 \vdash e_1 \Downarrow (\lambda \mathcal{E}_2, \cdot \mapsto \text{fun } x \rightarrow e) \quad \mathcal{E}_1 \vdash e_2 \Downarrow v_2 \quad \mathcal{E}_2[x \mapsto v_2] \vdash e \Downarrow v}{\mathcal{E}_1 \vdash e_1 e_2 \Downarrow v} \text{ APPE}$$

$$\frac{\mathcal{E}_1 \vdash e_1 \Downarrow (\lambda \mathcal{E}_2, f \mapsto \text{fun } x \rightarrow e) \quad \mathcal{E}_1 \vdash e_2 \Downarrow v_2 \quad \mathcal{E}_2[f \mapsto (\lambda \mathcal{E}_2, f \mapsto \text{fun } x \rightarrow e)] [x \mapsto v_2] \vdash e \Downarrow v}{\mathcal{E}_1 \vdash e_1 e_2 \Downarrow v} \text{ APPNAMEDE}$$

Let-Expressions

$$\frac{\mathcal{E}_1 \vdash e_1 \Downarrow v_1 \quad \mathcal{E}_1[x \mapsto v_1] \vdash e_2 \Downarrow v_2}{\mathcal{E}_1 \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow v_2} \text{ LETE} \quad \frac{\mathcal{E}_1[f \mapsto (\lambda \mathcal{E}_1, f \mapsto \text{fun } x \rightarrow e_1)] \vdash e_2 \Downarrow v_2}{\mathcal{E}_1 \vdash \text{let rec } f = \text{fun } x \rightarrow e_1 \text{ in } e_2 \Downarrow v_2} \text{ LETRECE}$$

By our syntactic restriction, the binding in a recursive let-expression is guaranteed to be an anonymous function.

Pattern Matching

$$\frac{}{v \text{ matches } _ \Rightarrow \emptyset} \text{ WILDM} \quad \frac{\text{x is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}} \text{ VARM} \quad \frac{}{\bullet \text{ matches } () \Rightarrow \emptyset} \text{ UNITM}$$

$$\frac{}{\top \text{ matches true} \Rightarrow \emptyset} \text{ TRUEM} \quad \frac{}{\perp \text{ matches false} \Rightarrow \emptyset} \text{ FALSEM} \quad \frac{\text{n is an integer pattern for } n}{n \text{ matches } n \Rightarrow \emptyset} \text{ INTM}$$

$$\frac{\text{s is an string pattern for } s}{n \text{ matches } s \Rightarrow \emptyset} \text{ STRINGM} \quad \frac{v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \quad \dots \quad v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{v_1, \dots, v_k \text{ matches } p_1, \dots, p_k \Rightarrow \mathcal{E}_1 \cup \dots \cup \mathcal{E}_k} \text{ TUPLEM}$$

$$\frac{\text{C is a constructor with no arguments}}{C \text{ matches } C \Rightarrow \emptyset} \text{ CONS0M} \quad \frac{\text{C is a constructor with arguments} \quad v \text{ matches } p \Rightarrow \mathcal{E}}{C(v) \text{ matches } C p \Rightarrow \mathcal{E}} \text{ CONS1M}$$

Match-Expressions

$$\frac{\mathcal{E} \vdash e_0 \Downarrow v_0 \quad \forall j < i (v_0 \text{ does not match } p_j) \quad v_0 \text{ matches } p_i \Rightarrow \mathcal{E}' \quad \mathcal{E}'' = \text{merge}(\mathcal{E}, \mathcal{E}') \quad \mathcal{E}'' \vdash e_i \Downarrow v}{\mathcal{E} \vdash \text{match } e_0 \text{ with } | p_1 \rightarrow e_1 \dots | p_k \rightarrow e_k \Downarrow v} \text{ MATCHE}$$

5 Extra Credit

Polymorphic Comparison

If you think about it, polymorphic comparison in OCaml doesn't really make sense. Equality can't possibly work the same way for *all* types. Whether or not polymorphic comparisons should exist in OCaml has been a long discussion in the OCaml community. We'll consider one facet of this problem. *The basic idea:* in OCaml we're not allowed to compare two function values:

```
utop # (fun x -> x) = (fun x -> x);;  
Exception: Invalid_argument "compare: functional value".
```

This is somewhat reasonable, e.g., it's unclear what the following should evaluate to:

```
(fun x -> x + 1) = (fun x -> 1 + x)
```

For a small amount of extra credit, update your interpreter so that it raises a `Compare_fun_val` exception when a comparison operator is applied to function values. Since this is extra credit, this is all we'll give you, but keep in mind that it's a *little* more subtle than it appears.

Your solution should not affect how the interpreter behaves on programs without comparisons of function values.

Type Errors

Providing useful type error messages for this mini-project is a lot trickier than in the last one. Since constraints are analyzed only *after* having gone through the entire expression, we have to make sure to maintain enough information to retrace what caused a constraint to fail. Also, although the order in which you process constraints doesn't affect the derived principal type, the order *does* affect which error is reported.

For extra credit, implement type error messages. You'll notice that a constraint has the following type:

```
type constr = ty * ty * (ty -> ty -> Error_msg.t)
```

The third value of a `constr` tuple is a function that generates an error message, should the given constraint ever fail. The challenge is to:

- make sure that every constraint has the correct error message generating function, with the correct position information;
- process constraints in the correct order, so that the right message is displayed.

In the case that you do not want to do the extra credit, you should carry no additional data in your constraints. If you don't do the extra credit, then the order in which you process constraints does not matter.

6 Putting Everything Together

After you've completed `type_of_expr` and `eval_expr` you should be able to run:

```
dune exec interp3 <filename>
```

to execute code you're written in another file. We've implemented a fragment of OCaml, so it should be easy to generate examples, e.g., `sum_of_squares.ml`:

```
let sum_of_squares x y =  
  let x_squared = x * x in  
  let y_squared = y * y in  
  x_squared + y_squared
```

Final Remarks

- There is a lot of repetition here, this is the nature of implementing a programming language. Even though there is a lot of code to write, it should go pretty quickly. Despite this, it's worthwhile to do a bit of code golfing, i.e., to think about how to implement the interpreter without too much code replication.
- Test along the way. Don't try to write the whole interpreter and test afterwards.
- You **must** use exactly the same names and types as given at the top of this file. They **must** appear in the file `interp3/lib/interp3.ml`. If you don't do this, we can't grade your submission. You are, of course, allowed to add your own functions.
- You're given a skeleton dune project to implement your interpreter. **Do not change any of the given code.** In particular, don't change the dune files or the utility files. When we grade your assignment we will be assume this skeleton code.

Good luck, happy coding.